

§ . 基础知识题 – 浮点数机内存存储格式(IEEE 754) 理解



要求:

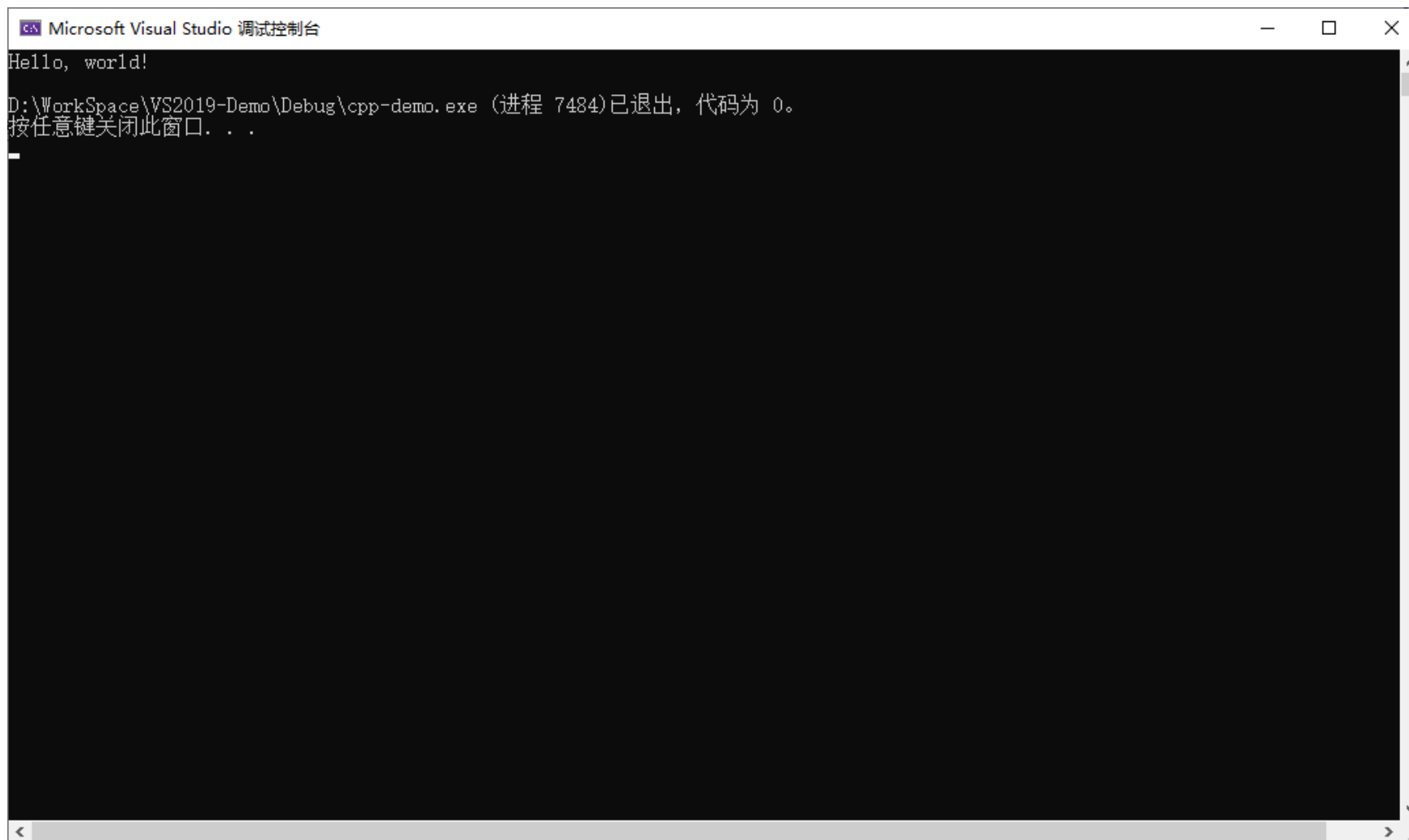
- 1、完成本文档中所有的题目并写出分析、运行结果
- 2、无特殊说明，均使用VS2022编译即可
- 3、直接在本文件上作答，**写出答案/截图（不允许手写、手写拍照截图）**即可；填写答案时，为适应所填内容或贴图，**允许调整**页面的字体大小、颜色、文本框的位置等
 - ★ 贴图要有效部分即可，不需要全部内容
 - ★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可
 - ★ **不允许**手写在纸上，再拍照贴图
 - ★ **允许**在各种软件工具上完成（不含手写），再截图贴图
- 4、转换为pdf后提交
- 5、**3月14日前**网上提交本次作业（在“文档作业”中提交）

§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

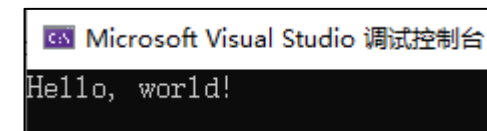


贴图要求：只需要截取输出窗口中的有效部分即可，如果全部截取/截取过大，则视为无效贴图

例：无效贴图

A screenshot of the Microsoft Visual Studio debug console. The window title is "Microsoft Visual Studio 调试控制台". The text content is: "Hello, world!", "D:\Workspace\VS2019-Demo\Debug\cpp-demo.exe (进程 7484)已退出, 代码为 0.", and "按任意键关闭此窗口. . .". The rest of the console area is a large black rectangle, indicating that the screenshot is too large and thus invalid.

例：有效贴图

A screenshot of the Microsoft Visual Studio debug console. The window title is "Microsoft Visual Studio 调试控制台". The text content is: "Hello, world!". This is a valid screenshot as it only captures the relevant output.



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

附：用WPS等其他第三方软件打开PPT，将代码复制到VS2022中后，如果出现类似下面的**编译报错**，则观察源程序编辑窗

的右下角是否为CR，如果是，单击CR，在弹出中选择CRLF，再次CTRL+F5运行即可

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello, World." << endl;
7      return 0;
8  }
9
```

100 % 未找到相关问题 行: 9 字符: 1 制表符 CR

输出

显示输出来源(S): 生成

1>—— 已启动生成: 项目: demo-CPP, 配置: Debug Win32 ——

1>demo.cpp

1>D:\WorkSpace\VS2019-demo\demo-CPP\demo.cpp(1,1): warning C4335: 检测到 Mac 文件格式: 请将源文件转换为 DOS 格式或 UNIX 格式

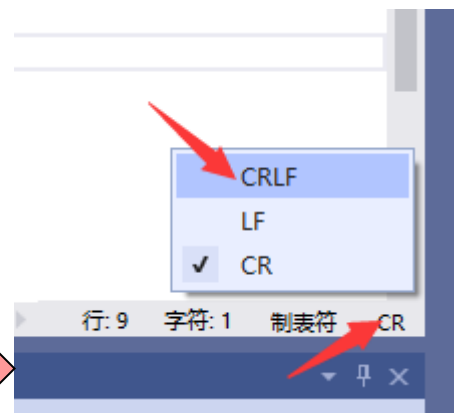
1>D:\WorkSpace\VS2019-demo\demo-CPP\demo.cpp(1,10): warning C4067: 预处理器指令后有意外标记 - 应输入换行符

1>MSVCRTD.lib(exe_main.obj) : error LNK2019: 无法解析的外部符号 _main, 函数 "int __cdecl invoke_main(void)" (?invoke_main@YAHXZ) 中

1>D:\WorkSpace\VS2019-demo\Debug\demo-CPP.exe : fatal error LNK1120: 1 个无法解析的外部命令

1>已完成生成项目 "demo-CPP.vcxproj" 的操作 - 失败。

输出 错误列表



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解



基础知识：用于看懂float型数据的内部存储格式的程序如下：

注意：除了对黄底红字的具体值进行改动外，其余部分不要做改动，也暂时不需要弄懂为什么（需要第6章的知识才能弄懂）

```
#include <iostream>
using namespace std;
int main()
{
    float f = 123.456;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

//注：忽略本题出现的warning

Microsoft
79
e9
f6
42

上例解读：单精度浮点数123.456，在内存中占四个字节，四个字节的值依次为0x42 0xf6 0xe9 0x79（按打印顺序逆向取）

转换为32bit则为：0100 0010 1111 0110 1110 1001 0111 1001

符号位

8位指数

23位尾数

§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解



基础知识：用于看懂double型数据的内部存储格式的程序如下：

注意：除了对黄底红字的具体值进行改动外，其余部分不要做改动，也暂时不需要弄懂为什么（需要第6章的知识才能弄懂）

```
#include <iostream>
using namespace std;
int main()
{
    double d = 1.23e4;
    unsigned char* p = (unsigned char*)&d;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    cout << hex << (int)*(p+4) << endl;
    cout << hex << (int)*(p+5) << endl;
    cout << hex << (int)*(p+6) << endl;
    cout << hex << (int)*(p+7) << endl;
    return 0;
}
```

Microsoft
0
0
0
0
6
c8
40

上例解读：双精度浮点数1.23e4，在内存中占八个字节，八个字节的值依次为0x40 0xc8 0x06 0x00 0x00 0x00 0x00 0x00(逆向)

转换为64bit则为：0100 0000 1100 1000 0000 0100 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

符号位

11位指数

52位尾数

§ . 基础知识题 – 浮点数机内存储格式(IEEE 754)理解



自学内容：自行以“IEEE754” / “浮点数存储格式” / “浮点数存储原理” / “浮点数存储方式”等关键字，在网上搜索相关文档，读懂并了解浮点数的内部存储机制

学长们推荐的网址：

<https://baike.baidu.com/item/IEEE%20754/3869922?fr=aladdin>

<https://zhuanlan.zhihu.com/p/343033661>

https://www.bilibili.com/video/BV1iW411d7hd?is_story_h5=false&p=4&share_from=ugc&share_medium=android&share_plat=android&share_session_id=e12b54be-6ffa-4381-9582-9d5b53c50fb3&share_source=QQ&share_tag=s_i×tamp=1662273598&unique_k=AuouMEO

https://blog.csdn.net/gao_zhennan/article/details/120717424

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

例: float型数的机内表示

格式要求: 多字节时, 每8bit中间加一个空格或- (例: "11010100 00110001" 或 "11010100-00110001")

例1: 100.25

下面是float机内存储手工转十进制的方法:

(1) 得到的32bit的机内表示是: 0100 0010 1100 1000 1000 0000 0000 0000 (42 c8 80 00)

(2) 其中: 符号位是 0

指数是 1000 0101 (填32bit中的原始形式)

指数转换为十进制形式是 133 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 6 (32bit中的原始形式按IEEE754的规则转换)

1000 0101

- 0111 1111

= 0000 0110 (0x06 = 6)

尾数是 100 1000 1000 0000 0000 0000 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.56640625 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.56640625 (加整数部分的1后)

100 1000 1000 0000 0000 0000 = $2^0 + 2^{-1} + 2^{-4} + 2^{-8}$

= 0.5 + 0.0625 + 0.00390625 = 0.56640625 => 加1 => 1.56640625

1.56640625 x 2^6 = 100.25 (此处未体现出误差)

下面是十进制手工转float机内存储的方法:

100 = 0110 0100 (整数部分转二进制为7位, 最前面的0只是为了8位对齐, 可不要)

0.25 = 01 (小数部分转二进制为2位)

100.25 = 0110 0100.01 = 1.1001 0001 x 2^6 (确保整数部分为1, 移6位)

符 号 位: 0

阶 码: 6 + 127 = 133 = 1000 0101

尾数(舍1): 1001 0001 => 1001 0001 0000 0000 0000 000 (补齐23位, 后面补14个蓝色的0)

100 1000 1000 0000 0000 0000 (从低位开始四位一组, 共23位)

注意:

- 1、作业中绿底/黄底文字/截图可不填
- 2、计算结果可借助第三方工具完成, 没必要完全手算

本页不用作答



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

例: float型数的机内表示

格式要求: 多字节时, 每8bit中间加一个空格或- (例: "11010100 00110001" 或 "11010100-00110001")

例2: 1.2

下面是float机内存储手工转十进制的方法:

(1) 得到的32bit的机内表示是: 0011 1111 1001 1001 1001 1001 1010 (3f 99 99 9a)

(2) 其中: 符号位是 0

指数是 0111 1111 (填32bit中的原始形式)

指数转换为十进制形式是 127 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 0 (32bit中的原始形式按IEEE754的规则转换)

0111 1111

- 0111 1111

= 0000 0000 (0x0 = 0)

尾数是 001 1001 1001 1001 1010 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.2000000476837158203125 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.2000000476837158203125 (加整数部分的1后)

001 1001 1001 1001 1010 = $2^{-3} + 2^{-4} + 2^{-7} + 2^{-8} + 2^{-11} + 2^{-12} + 2^{-15} + 2^{-16} + 2^{-19} + 2^{-20} + 2^{-22}$

= 0.125 + ... + 0.0000002384185791015625 (详见右侧蓝色) = 0.2000000476837158203125

=> 加1 = 1.2000000476837158203125 (此处已体现出误差)

下面是十进制手工转float机内存储的方法:

1 = 1 (整数部分转二进制为1位)

0.2 = 0011 0011 0011 0011 0011 0011 (小数部分无限循环, 转为二进制的24位)

=> 0011 0011 0011 0011 0011 010 (四舍五入为23位, 此处体现出误差)

1.2 = 1.0011 0011 0011 0011 0011 010 = 1.0011 0011 0011 0011 0011 010 x 2^0 (确保整数部分为1, 移0位)

符号位: 0

阶码: 0 + 127 = 127 = 0111 1111

尾数(舍1): 0011 0011 0011 0011 0011 010 (共23位)

001 1001 1001 1001 1001 1010 (从低位开始四位一组, 共23位)

注意:

- 1、作业中绿底/黄底文字/截图可不填
- 2、计算结果可借助第三方工具完成, 没必要完全手算

0.125 +
0.0625 +
0.0078125 +
0.00390625 +
0.00048828125 +
0.000244140625 +
0.000030517578125 +
0.0000152587890625 +
0.0000019073486328125 +
0.00000095367431640625 +
0.0000002384185791015625

0.2000000476837158203125

本页不用作答



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

A. 2351520.0251532 (此处设学号是1234567，需换成本人学号，小数为学号逆序，非本人学号0分，下同!!!)

注：尾数为正、指数为正

(1) 得到的32bit的机内表示是： 0100 1010 0000 1111 1000 0110 1000 0000 不是手算，用P.4方式打印)

(2) 其中：符号位是 0

指数是 1001 0100 (填32bit中的原始形式)

指数转换为十进制形式是 148 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 21 (32bit中的原始形式按IEEE754的规则转换)

尾数是 000 1111 1000 0110 1000 0000 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.1212921142578125 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.1212921142578125 (加整数部分的1)

注：转换为十进制小数用附加的工具去做，自己去网上找工具也行，但要满足精度要求(下同!!!)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

B. -251532.235152 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为正

(1) 得到的32bit的机内表示是： 1100 1000 0111 0101 1010 0011 0000 1111 (不是手算，用P.4方式打印)

(2) 其中：符号位是 1

指数是 1001 0000 (填32bit中的原始形式)

指数转换为十进制形式是 144 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 17 (32bit中的原始形式按IEEE754的规则转换)

尾数是 111 0101 1010 0011 0000 1111 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.91903865337371826171875 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.91903865337371826171875 (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

C. 0.00235152 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为负

(1) 得到的32bit的机内表示是： 0011 1011 0001 1010 0001 1011 1111 0101 (不是手算，用P.4方式打印)

(2) 其中：符号位是 0

指数是 0111 0110 (填32bit中的原始形式)

指数转换为十进制形式是 118 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 -9 (32bit中的原始形式按IEEE754的规则转换)

尾数是 001 1010 0001 1011 1111 0101 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.20397818088531494140625 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.20397818088531494140625 (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

D. -0.000251532 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为负

(1) 得到的32bit的机内表示是： 1011 1001 1000 0011 1110 0000 0000 1110 (不是手算，用P.4方式打印)

(2) 其中：符号位是 1

指数是 0111 0011 (填32bit中的原始形式)

指数转换为十进制形式是 115 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 12 (32bit中的原始形式按IEEE754的规则转换)

尾数是 000 0011 1110 0000 0000 1110 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.0302751064300537109375 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.0302751064300537109375 (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

A. 2351520.0251532 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为正

(1) 得到的64bit的机内表示是： 0100 0001 0100 0001 1111 0000 1101 0000 0000 0011 0011 1000 0011
1000 0101 0110 (不是手算，用P.5方式打印)

(2) 其中：符号位是 0

指数是 100 0001 0100 (填64bit中的原始形式)

指数转换为十进制形式是 1044 (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 21 (64bit中的原始形式按IEEE754的规则转换)

尾数是 0001 1111 0000 1101 0000 0000 0011 0011 1000 0011 1000 0101 0110 (填64bit中的原始形式)

尾数转换为十进制小数形式是 0.121292126251792975466514690197072923183441162109375
(64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.121292126251792975466514690197072923183441162109375
(加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

B. -251532.235152 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为正

(1) 得到的64bit的机内表示是： 1100 0001 0000 1110 1011 0100 0110 0001 1110 0001 1001 0111 0101 1111 0010 1101 (不是手算，用P.5方式打印)

(2) 其中：符号位是 1

指数是 100 0001 0000 (填64bit中的原始形式)

指数转换为十进制形式是 1040 (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 17 (64bit中的原始形式按IEEE754的规则转换)

尾数是 1110 1011 0100 0110 0001 1110 0001 1001 0111 0101 1111 0010 1101 (填64bit中的原始形式)

尾数转换为十进制小数形式是 0.9190386593017578764630570731242187321186065673828125

(64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.9190386593017578764630570731242187321186065673828125

(加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

C. 0.00235152 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为负

(1) 得到的64bit的机内表示是： 0011 1111 0110 0011 0100 0011 0111 1110 1010 1111 1101 1110 0101 0100 1000 1010 (不是手算，用P.5方式打印)

(2) 其中：符号位是 0

指数是 011 1111 0110 (填64bit中的原始形式)

指数转换为十进制形式是 1014 (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 -9 (64bit中的原始形式按IEEE754的规则转换)

尾数是 0011 0100 0011 0111 1110 1010 1111 1101 1110 0101 0100 1000 1010 (填64bit中的原始形式)

尾数转换为十进制小数形式是 0.203978240000000088372189566143788397312164306640625

(64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.203978240000000088372189566143788397312164306640625

(加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

D. -0.000251532 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为负

(1) 得到的64bit的机内表示是： 1011 1111 0011 0000 0111 1100 0000 0001 1011 0110 1100 0001 1111 1100 0001 0111 (不是手算，用P.5方式打印)

(2) 其中：符号位是 1

指数是 011 1111 0011 (填64bit中的原始形式)

指数转换为十进制形式是 1011 (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 -12 (64bit中的原始形式按IEEE754的规则转换)

尾数是 0000 0111 1100 0000 0001 1011 0110 1100 0001 1111 1100 0001 0111
(填64bit中的原始形式)

尾数转换为十进制小数形式是 0.0302750720000000139719986691488884389400482177734375
(64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.0302750720000000139719986691488884389400482177734375
(加整数部分的1)

§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

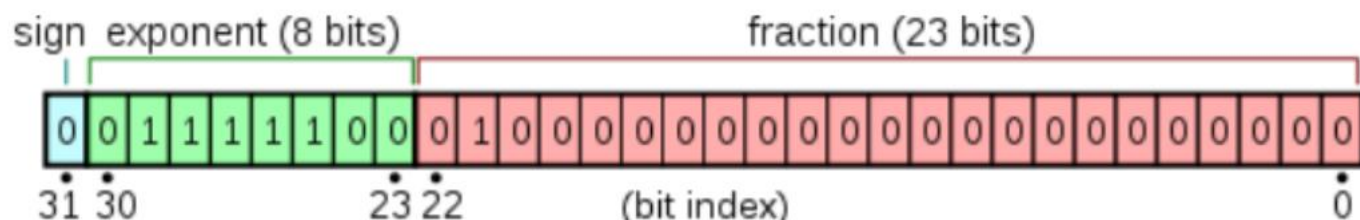


3、总结

(1) float型数据的32bit是如何分段来表示一个单精度的浮点数的？给出bit位分段解释

尾数的正负如何表示？尾数如何表示？指数的正负如何表示？指数如何表示？

如图，float型数据一共占4字节，共32位。第31位（最高位）为符号位，0表示正数，1表示负数。第30位到第23位（共8位）表示偏移后的指数，第22位到第0位（共23位）表示尾数。



尾数的正负由第31位表示，0表示正数，1表示负数。在以二进制格式存储十进制浮点数时，首先要把十进制浮点数表示为二进制格式。以十进制浮点数20.5举例，十进制浮点数20.5（绝对值） = 二进制10100.1。然后，需要把这个二进制数转换为以2为底的指数形式：二进制10100.1 = 1.01001 * 2⁴。注意转换时，对于乘号左边的那个二进制数1.01001，需要把小数点放在左起第一位和第二位之间。且第一位需要是个非0数。这样表示好之后，其中的1.01001就是尾数。（用二进制数表示十进制浮点数时，表示为尾数*指数的形式，并把尾数的小数点放在第一位和第二位之间，然后保证第一位数非0，这个处理过程叫做规范化）因为二进制只有0和1两个数值，所以第一位数只能是1。对尾数进行隐藏高位1和低位补0补齐23位后，就得到了23位尾数。

指数即十进制数转换为二进制浮点数再规范化后2的指数（如上例中4为指数）。指数在float型数据中以偏移后的指数表示，这是一个8位二进制无符号数，因此不用表示正负。float型的偏移量为127（即2⁷-1），对原指数加上127后就得到了一个非负数，将其转换为二进制数，再高位补0补齐8位后，就得到了8位偏移后的指数。如上例中，原指数为4，加上127后为131，转换为8位二进制数为1000 0011，即为偏移后指数。



(2) 为什么float型数据只有7位十进制有效数字？为什么最大只能是 3.4×10^{38} ？
有些资料上说有效位数是6~7位，能找出6位/7位不同的例子吗？

float型数据的有效位数由尾数决定，23位尾数能表示的最大数字为 $2^{23}-1 = 8388607$ ， $10^6 < 8388607 < 10^7$ ，所以说float型数据的有效位数是7位。

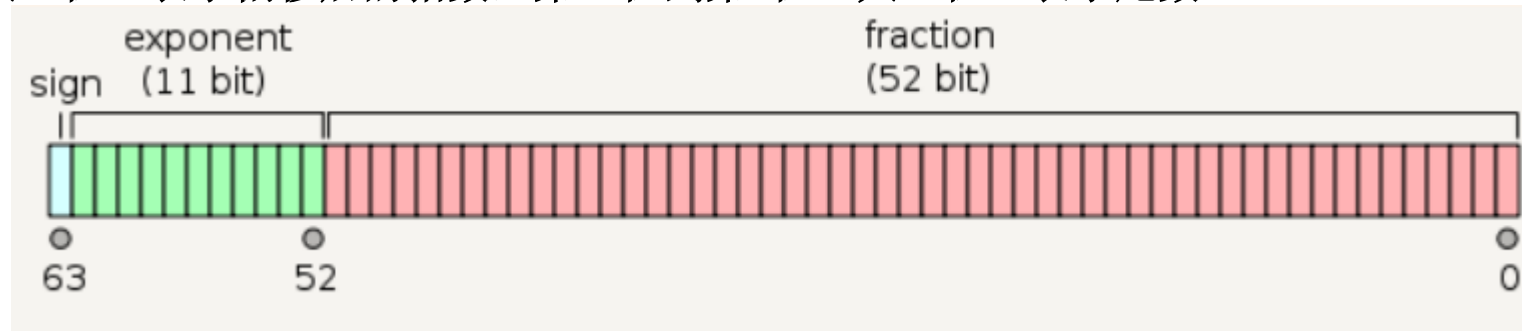
当尾数和指数全为1时，表示的最大值为 $1.11\dots$ (共23位) $\times 2^{128}$ ，约等于 3.4×10^{38} 。

例子：1.325441用float型存储时，能保证7位有效数字1.325441相同；
1.998524用float型存储时，只能保证6位有效数字1.99852相同，第七位不相同。



(3) double型数据的64bit是如何分段来表示一个双精度的浮点数的？给出bit位的分段解释
尾数的正负如何表示？尾数如何表示？指数的正负如何表示？指数如何表示？

如图，double型数据一共占8字节，共64位。第63位（最高位）为符号位，0表示正数，1表示负数。第62位到第52位（共11位）表示偏移后的指数，第51位到第0位（共52位）表示尾数。



尾数的正负由第63位表示，0表示正数，1表示负数。在以二进制格式存储十进制浮点数时，首先需要把十进制浮点数表示为二进制格式。以十进制浮点数20.5举例，十进制浮点数 $20.5 = \text{二进制}10100.1$ 。然后，需要把这个二进制数转换为以2为底的指数形式： $\text{二进制}10100.1 = 1.01001 * 2^4$ 。注意转换时，对于乘号左边的那个二进制数1.01001，需要把小数点放在左起第一位和第二位之间，且第一位需要是个非0数。这样表示好之后，其中的1.01001就是尾数。（用二进制数表示十进制浮点数时，表示为尾数*指数的形式，并把尾数的小数点放在第一位和第二位之间，然后保证第一位数非0，这个处理过程叫做规范化）因为二进制只有0和1两个数值，所以第一位数只能是1。对尾数进行隐藏高位1和低位补0补齐52位后，就得到了52位尾数。

指数即十进制数转换为二进制浮点数再规范化后2的指数（如上例中4为指数）。指数在double型数据中以偏移后的指数表示，这是一个11位二进制无符号数，因此不用表示正负。double型的偏移量为1023（即 $2^{10}-1$ ），对原指数加上1023后就得到了一个非负数，将其转换为二进制数，再高位补0补齐11位后，就得到了11位偏移后的指数。如上例中，原指数为4，加上1023后为1027，转换为11位二进制数为100 0000 0011，即为偏移后指数。



- (4) 为什么double型数据只有15位十进制有效数字？为什么最大只能是 1.7×10^{308} ？
有些资料上说有效位数是15~16位，能找出15位/16位不同的例子吗？

double型数据的有效位数由尾数决定，52位尾数能表示的最大数字为 $2^{52}-1 = 4503599627370495$ ，
 $10^{15} < 4503599627370495 < 10^{16}$ ，所以double型数据的有效位数是15位。

当尾数和指数全为1时，表示的最大值为 $1.11... (共52位) \times 2^{1024}$ ，约等于 1.7×10^{308} 。

例子：125.2646464645251用double型存储时，能保证16位有效数字125.2646464645251相同；
125.2646464645255用double型存储时，只能保证15位有效数字1.99852相同，第16位不相同。



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

4、思考

(1) 8/11bit的指数的表示形式是2进制补码吗？如果不是，一般称为什么方式表示？

不是。一般称为移码表示，即原指数加上偏移量127/1023后转换的8/11位二进制数。

(2) double赋值给float时，下面两个程序，double型常量不加F的情况下，左侧有warning，右侧无warning，为什么？

总结一下规律

左边double型常量1.2的浮点数表示是0 0111111111 0011 00110011 00110011 00110011 00110011 00110011 00110011。尾数在23位之后仍然有数字，而float型的尾数只有23位，导致double型赋值给float型时尾数23位后面的数字被截断，使得精度丢失，所以有warning。而右边100.25的浮点数表示中的尾数为10010001（后面全是0），这样在赋给float型时尾数是相同的，不存在数字丢失的情况，所以没有warning。

总结：当double型数据的尾数部分有效数字不超过23位时，且偏移前的指数不超过127时，赋值给float型时不会有warning；当double型数据的尾数部分有效数字超过23位时，或偏移前的指数超过127时，赋值给float型时会有warning。

```
#include <iostream>
using namespace std;
int main()
{
    float f = 1.2;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

warning C4305: “初始化”: 从“double”到“float”截断

```
#include <iostream>
using namespace std;
int main()
{
    float f = 100.25;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

